

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.9.27	完成日期	2024.9.27

第一题

1. 实验题目

Delete Duplicate

2. 实验目的

已知线性表中的元素以递增有序排列, 并以单链表作存储结构。试写一个高效的算法, 删除表中所有冗余的结点, 即数据域相同的结点只保留一个。

3. 算法设计

1. 防止头指针为空

在函数开始时, 首先判断 head 是否为空指针。若为空, 直接返回, 因为没有需要处理的链表。

2. 通过 data 值判断下一个节点是否冗余

使用 while 循环遍历链表, 依次判断当前节点 p 和下一个节点 p->next 的 data 值是否相同。如果相同, 说明下一个节点是冗余的, 需要被删除。

3. 先存储冗余节点，后释放其内存

若发现冗余节点，首先将该节点保存在 `tmp` 中，然后将 `p->next` 指向冗余节点的下一个节点，这样就跳过了冗余节点。接着，使用 `delete` 释放冗余节点的内存。

4. 使得冗余节点的上一节点的 `next` 指向冗余节点的下一节点

上一步中，已经修改了当前节点 `p` 的 `next` 指向冗余节点的下一个节点，从而删除了冗余节点，使得链表结构正确。

5. 对所有节点进行遍历，重复上述操作

遍历链表中的所有节点，直到链表的末尾。每遇到冗余节点就进行删除，直到没有冗余节点为止。

4. 程序运行与测试

```
void delete_duplicate(LinkList &head)
{
    if(!head)//空指针
        return;
    LinkNode* p=head;
    while(p->next)
    {
        if(p->next->data==p->data)//下一个结点是冗余
        {
            LinkNode* tmp=p->next;//存储结点
            p->next=p->next->next;
            delete tmp;//释放内存
        }
        else
            p=p->next;//更新 p
    }
}
```

5. 实验总结与心得

1.代码只需要实现一个函数，具体实现不需要 `main` 函数。

本实验要求我们只实现链表去重的核心功能，因此不需要完整的程序结构，只需要实现 `delete_duplicate` 函数即可。

2.需要包含.h 头文件

在代码中，应该包含必要的头文件，特别是`#include <iostream>`等标准库头文件，另外需要包含定义链表节点的头文件（如 `LinkList.h`），以确保函数能够正常编译和运行。

3.需要释放冗余节点的内存

在删除冗余节点时，务必使用 `delete` 操作符释放冗余节点的内存，防止内存泄漏。

第二题

1. 实验题目

Insert for single link list with head node

2. 实验目的

实现如下函数：

```
void List::insert(int toadd, int pos)
```

3. 算法设计

1.防止头结点为空判断

在函数开始时，判断 **head** 是否为空指针。如果链表为空，则直接返回，避免后续操作引发错误。

2.利用整型 **pos** 找到插入位置的前一个结点

通过一个循环，用变量 **p** 从 **head** 开始遍历链表，向后移动 **pos-1** 次，找到插入位置的前一个结点。在移动过程中，若发现 **p** 为空，则提前返回，防止越界或访问非法内存。

3.创建新结点

在找到插入位置的前一个结点后，动态分配内存创建一个新结点 **tmp**，将要插入的值 **toadd** 赋给 **tmp->data**。

4.接入新结点

将新结点 **tmp** 插入到链表中，使其连接到正确的位置。具体步骤是让 **tmp->next** 指向原来前一个结点的 **next**，然后更新前一个结点的 **next** 指向新结点 **tmp**。

4.程序运行与测试

```
void List::insert(int toadd, int pos)
{
    if(!head)
        return;
    ListNode* p=head;
    for(int i=pos;i>1;i--)
    {
        p=p->next;
        if(!p)
            return;
    }
    //找到 pos 个结点的前一个位置
```

```
ListNode* tmp=new ListNode;//创建新结点
tmp->data=toadd;
tmp->next=p->next;
p->next=tmp;
}
```

5.实验总结与心得

1.利用 pos 找到插入的位置

插入操作的核心是准确找到插入位置的前一个结点。在代码中，使用循环控制 p 的移动，直至到达正确的前置位置。

2.防止找到的 p 为空

在遍历过程中，必须判断是否超出了链表范围。如果 p 为空，直接返回，避免出现非法操作或逻辑错误。

3.内存分配与链表操作的结合

通过动态分配内存实现新结点的创建，并通过指针操作保证链表结构的正确性。要注意每一步指针操作的顺序，确保链表不被破坏。

4.边界条件处理

对空链表、无效位置（如位置超出范围）的特殊情况进行了处理，提高了程序的鲁棒性。

第三题

1.实验题目

Loops in the Linked List

2.实验目的

给定链表结点类型定义，要求实现 `check` 函数返回对于给定链表是否存在环，注意链表的边界情况。

3.算法设计

1.防止链为空

在函数开始时，判断 `head` 是否为空指针。如果链表为空，则直接返回 `false`，因为空链表没有环。

2.设定快慢指针

定义两个指针：一个为慢指针 `slow`，初始化指向链表的头节点；另一个为快指针 `fast`，初始化指向链表的第二个节点。

3.快指针一次走两步，慢指针一次走一步

在遍历链表时，慢指针每次向前移动一步，而快指针每次向前移动两步。通过这种方式，快指针会在环形链表中追赶慢指针。

4.当快指针遇上慢指针，说明链存在环

如果链表中存在环，快指针和慢指针终会在某一时刻相遇，此时返回 `true`。若快指针或快指针的下一步为 `NULL`，说明链表无环，返回 `false`。

4.程序运行与测试

```
bool check(node *head) {  
    if (!head) //链表为空  
        return false;  
  
    node *slow = head; //慢指针  
    node *fast = head->next; //快指针 在第二个位置  
  
    while (fast && fast->next) {  
        if (slow == fast)  
            return true;  
        slow = slow->next; // 慢指针每次走一步  
        fast = fast->next->next; // 快指针每次走两步  
    }  
    return false;  
}
```

5.实验总结与心得

1.需要利用好快走慢走的快慢指针

快慢指针是判断链表是否有环的核心技巧。通过快指针和慢指针的不同步速率，可以有效检测环的存在。

2.若不存在环，则慢指针追不上快指针；若存在环，则快慢指针终会相遇

在无环链表中，快指针最终会到达 NULL 或 NULL->next，程序返回 false；在有环链表中，由于快慢指针的移动方式不同，快指针会在环中不断追赶慢指针，并最终相遇。

3.算法时间复杂度与空间复杂度

4.时间复杂度：O(n)，其中 n 是链表的节点数。快指针最多遍历链表的两倍长度。

空间复杂度：O(1)，不需要额外的数据结构，只使用了两个指针变量。

5.边界条件处理

考虑了空链表和单节点链表的特殊情况，通过 `fast` 和 `fast->next` 的判断保证程序的安全性。